E7: Introduction to Computer Programming
Midterm 2A

Thursday, October 26

Full Name

Student ID Number

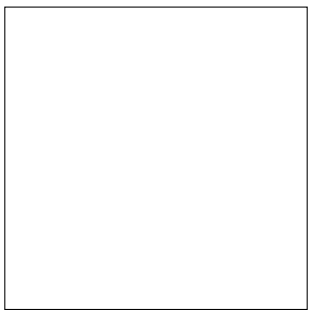
Instructions:

- You have 1 hour and 20 minutes to complete this exam.
- The exam is closed book, closed notes, closed computer, closed calculator.
- You may only use one 8.5" × 11" sheet of notes (both sides). No other resources are permitted.
- Mark your answers on the exam paper within the designated spaces. Responses written on scratch paper or outside of the specified answer spaces will not be graded.
- This exam is DOUBLE-SIDED. Make sure to read both sides of each page.
- The number of points associated with each part of each question is indicated.
- The exam is worth a total of 100 points.
- For questions with circular bubbles, you should select exactly *one* choice.
  - ◯ You may only chose one option!
  - ◯ Or this one, but not both!
  - ◯ Or this one, but no more than one!
- For questions with square boxes, you may select *multiple* choices.
  - ☐ You can select this one!
  - ☐ You can select this one too!
  - ☐ You can select one, two, or all three of these!
- Only include your FINAL answers in the designated boxes. Any additional text will count as part of your answer and may result in a reduced grade.

Good luck!

"We're rooting for you! Draw a smiley face in the box and collect 1 free point :)"

"All responses will receive 1 point. Don't spend too much time here."



Question 1: Starters (25 points)

Each part below is independent of the other.

(a) (2 pt) What is the second programming language that was introduced in E7 this semester? For questions with circular bubbles, you should select exactly *one* choice.

- ◯ JavaScript
- ● MATLAB
- ◯ LOLCODE
- ◯ C++
- ◯ Lisp
- ◯ Snap

(b) (1 pt) Which of the following statements will be ignored by the MATLAB interpreter?

- ◯ `x = 25`
- ◯ `x = @disp`
- ◯ `x = 25;`
- ● `% x = 25`
- ◯ `'x = 25'`
- ◯ None of the options

(c) (5 pt) Select the expression(s) that can be used to calculate: $\sqrt{\frac{1 - sin(2\pi)}{\pi + 4}}$ in MATLAB. For questions with square boxes, you may select *multiple* choices.

- ☐ `sqrt((1 - sin(2 * pi)) / pi + 4)`
- ■ `((1 - sin(2 * pi)) / (pi + 4)) ^ 0.5`
- ☐ `np.sqrt((1 - np.sin(2 * np.pi)) / (np.pi + 4))`
- ☐ `sqrt((1 - sin(2pi)) / (pi + 4))`
- ☐ `((1 - sin(2 * pi)) / (pi + 4)) ^ 1 / 2`
- ■ `((1 - sin(2 * pi)) / (pi + 4)) ^ (1 / 2)`
- ■ `sqrt((1 - sin(2 * pi)) / (pi + 4))`
- ☐ `((1 - sin(2 * pi)) / (pi + 4)) ** 0.5`
- ☐ `sqrt(1 - sin(2 * pi)) / (pi + 4)`
- ☐ None of the options

(d) (4 pt) The function below is written using Python syntax. Redefine the same function using MATLAB syntax. Assume that `t` and `w` are two scalar variables and that `r` is of type bool/logical. Your answer should be one line of code.

```
result = lambda t, w, r : (t > 45) and (w < 10) or not r
```

```
result = @(t, w, r) (t > 45) && (w < 10) || ~ r

% or result = @(t, w, r) (t > 45) & (w < 10) | ~ r
```

(e) (4 pt) Which expression(s) result in the following array in MATLAB: `[-2 1 4 7 10 13]`

- ☐ `-2:3:16`
- ■ `-2:3:13`
- ☐ `(-2, 1, 4, 7, 10, 13)`
- ■ `[-2, 1, 4, 7, 10, 13]`
- ☐ `linspace(-2, 13, 4)`
- ☐ `linspace(-2, 14, 7)`
- ■ `linspace(-2, 13, 6)`
- ☐ `-2:13`
- ☐ `13:-3:-2`
- ☐ `0:3:13`

Parts (f), (g), (h), and (i) are all related to the following MATLAB script:

```
A = [1, 3, 5, 7, 9];
B = [8, 6, 4, 2, 0];
C = A > B;
A(C) = 2;
```

Assume the script above has been executed. For each part, write the output that will be displayed when the expression is evaluated. If an error occurs, write "Error". If nothing is displayed, write "Nothing". Write your answers within the designated space for each part.

(f) (2 pt) `disp(C)`

```
[0 0 1 1 1]
```

(g) (2 pt) `disp(A)`

```
[1 3 2 2 2]
```

(h) (2 pt) `disp(B(end))`

```
0
```

(i) (2 pt) `disp(B(4:-1:2))`

```
[2 4 6]
```

Question 2: Why Computers Complicate Numbers!? (10 points)

The following might be helpful for some of the next parts:

$2^2 = 4$, $2^3 = 8$, $2^4 = 16$, $2^5 = 32$, $2^6 = 64$, $2^7 = 128$, $2^8 = 256$.

$3^2 = 9$, $3^3 = 27$, $3^4 = 81$.

$4^2 = 16$, $4^3 = 64$, $4^4 = 256$.

$5^2 = 25$, $5^3 = 125$, $5^4 = 625$.

(a) (3 pt) What decimal number corresponds to the following signed binary integer: `10100001`

-33

(b) (2 pt) Convert the decimal number 128 into quinary (base-5).

1003

(c) (2 pt) Select the base(s) for which the number `1230` is a valid number.

- ☐ Base 2
- ☐ Base 3
- ■ Base 4
- ■ Base 5
- ■ Base 6
- ■ Base 7
- ■ Base 8
- ■ Base 9

(d) (1 pt) What is the binary number that immediately precedes (is smaller than) the following binary integer: `00101000`. Your answer should be in the form of another binary number.

00100111

(e) (2 pt) Why does this expression evaluate to `False` in Python? Select ALL that apply.

```
0.1 + 0.1 + 0.1 == 0.3
```

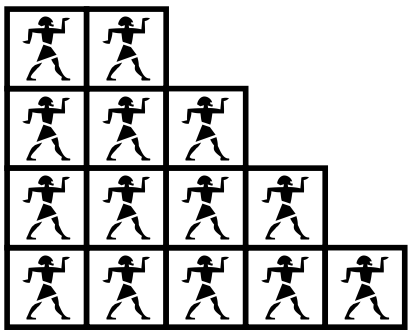
- ☐ Python's addition operator has a bug when working with floating-point numbers
- ☐ Python uses single precision instead of double to represent floating-point numbers
- ■ Not all numbers can be exactly represented using a finite sum of powers of 2
- ■ Accumulation of round-off errors
- ☐ Python represents decimal numbers precisely, so the expression should evaluate to `True`
- ☐ Python employs a specific floating-point format that always results in small inaccuracies
- ☐ All floating-point numbers in Python are stored as approximations

Question 3: Pyramids, Again (10 points)

I encouraged you to go over Midterm 1. So, this is a modified question from Midterm 1. Note that the question is similar but NOT exactly the same. Write a function `pyramids()` which takes three positive integers: the number of available building blocks (`n`), the number of blocks that should be used for the top layer (`t`), and the number of blocks that should be added to each subsequent layer (`step`). The function returns two values: the number of layers in the pyramid (`layers`), and the number of blocks that remain unused after constructing the pyramid. When constructing the pyramid, the top layer must contain `t` blocks, and each subsequent layer should have `step` more block(s) than the layer directly above it. So, if you start with `n = 16` blocks and need to use `t = 2` blocks for the top layer and `step = 1` more block for each subsequent layer, then you can construct 4 layers ($2 + 3 + 4 + 5 = 14$ blocks) and will have 2 unused blocks.

Test Cases (MATLAB Syntax):

```
>> [layers, n] = pyramids(16, 2, 1)
layers =
     4
n =
     2
```



```
>> [layers, n] = pyramids(16, 2, 2)
layers =
     3
n =
     4

>> [layers, n] = pyramids(16, 2, 10)
layers =
     2
n =
     2
```

Anything written on this page will NOT be graded. Only answers within the designated boxes on the next page will be graded. The function below is written using MATLAB syntax.

```
function [layers, n] = pyramids(n, t, step)

______(a)______;

blocks_current_layer = ______(b)______; % blocks needed for the current layer

while ______(c)______

    ______(d)______;

    ______(e)______;

    blocks_current_layer = ______(f)______;

end
end
```

You will implement the function `pyramids()` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use ANY functions or methods, built-in or otherwise.

(a) (2 pt) Fill in blank (a). Use MATLAB syntax for all parts on this question.

```
layers = 0
```

(b) (1 pt) Fill in blank (b).

```
blocks_current_layer = t
```

(c) (2 pt) Fill in blank (c).

```
while n >= blocks_current_layer
% or while n - blocks_current_layer >= 0
% or while n >= t + layers * step
```

(d) (1 pt) Which of these could fill in blank (d)?

- ○ `layers + 1`
- ○ `layers = 0`
- ○ `layers == layers + 1`
- ● `layers = layers + 1`
- ○ `layers = 2`
- ○ `layers = t`
- ○ `layers = 0`
- ○ `layers = layers + step`

(e) (2 pt) Fill in blank (e).

```
n = n - blocks_current_layer
```

(f) (2 pt) Fill in blank (f).

```
blocks_current_layer = blocks_current_layer + step
% or blocks_current_layer = t + layers * step
```

Question 4: Pythonista and MATLAB: Code SOS (15 points)

Rainfall measurements are typically collected at multiple stations, with each station corresponding to a specific area. Each station is associated with a rainfall measurement, denoted R_i, and the area it covers, denoted A_i, where i is the station number ($i = 1, 2, ..., n$). You have been provided with two 1D arrays: one containing the rainfall measurements (`R`) and the other containing the respective station areas (`A`) in square miles. The following built-in MATLAB functions might be helpful.

Operation	Function
Sum of all elements	`sum()`
Maximum of all elements	`max()`

Operation	Function
Minimum of all elements	`min()`
Average of all elements	`mean()`

(a) (2 pt) Using MATLAB syntax and *one line of code*, write an expression that extracts all elements of `R` that are greater than the mean of the rainfall measurements. Assign this expression to a variable called `above_mean`.

```
% Assume R is already defined

above_mean = R(R > mean(R))
```

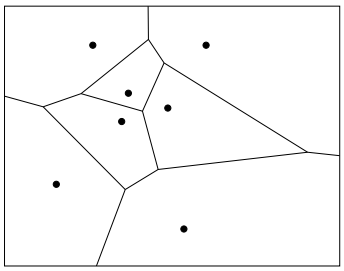
(b) (3 pt) Using MATLAB syntax and *one line of code*, write an expression that computes the number of stations that had rain greater than zero and an area greater than 2 square miles. Assign this expression to a variable called `events`.

```
% Assume R and A are already defined

events = sum(R > 0 & A > 2)
```

(c) (2 pt) Instead of taking the mean of all measurements, it is common to assign weights to each station based on the area surrounding it and to calculate a weighted mean rainfall ($\overline{R}$) as:

$$\overline{R} = \frac{\sum_{i=1}^{n} A_i R_i}{\sum_{i=1}^{n} A_i} = \frac{A_1 R_1 + A_2 R_2 + ... + A_n R_n}{A_1 + A_2 + ... + A_n}$$



Which of the following expressions can be used to compute $\overline{R}$ in MATLAB?

- ◯ `sum(A * R) / sum(A)`
- ◯ `sum(AR) / sum(A)`
- ● `sum(A .* R) / sum(A)`
- ◯ `sum((A .* R) ./ A)`
- ◯ `sum(A * R / A)`
- ◯ None of the options

Parts (d) and (e) are independent from one another and from the previous parts.

(d) (5 pt) Pythonista Smith, who is well-versed in Python, is finding the transition to MATLAB challenging. They've written a MATLAB function and are reaching out to identify any errors. The code includes line numbers on the left. You can assume that `rain` is a 1D array. What is wrong with the following MATLAB function? Select ALL that apply. You should select all options that would result in an error when running the function.

```
1  function [rain_total, rain_mean] = rain_stats(rain)
2      rain_total = 0;
3      count_days = 0;
4      for i = 0:length(rain)
5          if rain[i] == -999
6              break
7          elseif rain[i] >= 0
8              rain_total = rain_total + rain[i];
9              count_days = count_days + 1;
10     rain_mean = rain_total / count_days;
11     return rain_total, rain_mean
12 end
```

- ■ The `return` statement on line 11
- ☐ Using `>=` on line 7
- ☐ Ending assignment statements with `;`
- ☐ Using `break` on line 6
- ☐ Using indentation
- ■ Missing at least one `end` keyword
- ■ Iterating over `0:length(rain)`
- ■ Indexing `rain` using square brackets

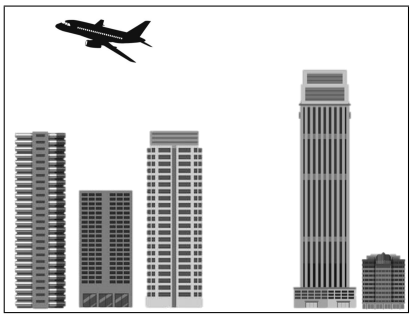
(e) (3 pt) Consider the following MATLAB script. How many `end` keywords should be added for this script to run without any errors?

```
force = [25, 22, 10, 10, 1, 25];
for i = force
if i > 5
while i < 50
i = i * 2;
elseif i > 10
i = i + 2;
```

- ○ 0
- ○ 1
- ○ 2
- ● 3
- ○ 4
- ○ 5
- ○ 6
- ○ An error will occur regardless of `end`

Question 5: Tallest Building (15 points)

The City of San Francisco wants to improve its urban design. They have street images and need an automated method to determine the tallest building. They already have an algorithm that converts a street image into a 2D array where each column in the array corresponds to a horizontal position in the street. Each cell in the array is marked with a `1` if an object (like part of a building or an airplane) is detected at that position, and a `0` if nothing is detected, as demonstrated below.



```
[[0 1 0 0 0 0],
 [0 0 0 0 1 0],
 [1 0 1 0 1 0],
 [1 1 1 0 1 0],
 [1 1 1 0 1 1]]
```

Buildings are represented by `1`'s depending on their height. The cell directly above the top of a building is always correctly represented by `0`. However, objects above this can be incorrectly represented by `1`, which should be ignored. Write a Python function `tallest()` which takes a 2D NumPy array `data` and returns the height of the tallest building `max_height` in feet. Each row in `data` corresponds to a height of 20 feet. Anything written on this page will NOT be graded.

```
def tallest(data):
    """Return the height of the tallest building.
    >>> tallest(np.array([[0, 1, 0, 0, 0, 0],
                          [0, 0, 0, 0, 1, 0],
                          [1, 0, 1, 0, 1, 0],
                          [1, 1, 1, 0, 1, 0],
                          [1, 1, 1, 0, 1, 1]]))
    80
    """
    max_height = 0 # maximum height
    rows, columns = data.shape # shape returns a tuple of array dimensions
    for ______(a)______:
        height = 0 # initialize height
        for ______(b)______:
            if ______(c)______ == 1:
                height += ______(d)______
            else:
                ______(e)______
        if ______(f)______:
            ______(g)______
    return max_height
```

You will implement the function `tallest()` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not use ANY built-in functions or methods. You may not modify the level of indentation of any line.

(a) (2 pt) Which of these could fill in blank (a)?

- ○ `row in range(rows - 1, -1, -1)`
- ○ `row in range(rows, 0, -1)`
- ○ `row in range(rows)`
- ○ `row in rows`
- ○ `column in range(columns + 1)`
- ○ `column in range(columns - 1)`
- ● `column in range(columns)`
- ○ `column in columns`

(b) (2 pt) Which of these could fill in blank (b)?

- ● `row in range(rows - 1, -1, -1)`
- ○ `row in range(rows, 0, -1)`
- ○ `row in range(rows)`
- ○ `row in rows`
- ○ `column in range(columns + 1)`
- ○ `column in range(columns - 1)`
- ○ `column in range(columns)`
- ○ `column in columns`

(c) (2 pt) Fill in blank (c).

```
if data[row, column] == 1:
```

(d) (2 pt) Fill in blank (d).

```
height += 20
```

(e) (2 pt) Fill in blank (e).

```
break
```

(f) (3 pt) Fill in blank (f).

```
if height > max_height:
```

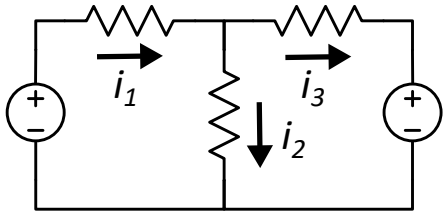
(g) (2 pt) Fill in blank (g).

```
max_height = height
```

Question 6: Ohm My Goodness! (15 points)

An electric circuit has three resistors as shown below. The currents in each resistor are unknown, and are represented as: i_1, i_2, i_3. Using Ohm's law, you developed the following system of linear equations, which you want to write in matrix form $\mathbf{A} \cdot \mathbf{i} = \mathbf{b}$:

$$\begin{cases} 2i_1 + 8i_2 = 10 \\ 4i_3 - 8i_2 = 6 \\ i_1 + 2i_3 = 0 \end{cases} \rightarrow \mathbf{A} \times \begin{bmatrix} i_1 \\ i_2 \\ i_3 \end{bmatrix} = \begin{bmatrix} 10 \\ 6 \\ 0 \end{bmatrix}$$



(a) (2 pt) Which of the following is the correct representation of matrix $\mathbf{A}$?

- ● $\begin{bmatrix} 2 & 8 & 0 \\ 0 & -8 & 4 \\ 1 & 0 & 2 \end{bmatrix}$
- ○ $\begin{bmatrix} 2 & 8 & 10 \\ 4 & -8 & 6 \\ 1 & 2 & 0 \end{bmatrix}$
- ○ $\begin{bmatrix} 2 & 8 \\ 4 & -8 \\ 1 & 2 \end{bmatrix}$
- ○ $\begin{bmatrix} 2 & 8 & 0 \\ 0 & 4 & -8 \\ 1 & 0 & 2 \end{bmatrix}$
- ○ $\begin{bmatrix} 2 & 8 & 0 \\ 0 & -8 & 4 \\ 1 & 2 & 3 \end{bmatrix}$
- ○ $\begin{bmatrix} 2 & 8 & 10 \\ 4 & -8 & 6 \\ 1 & 2 & 0 \end{bmatrix}$

(b) (2 pt) Using MATLAB syntax and *one line of code*, define matrix $\mathbf{A}$.

```
A = [2 8 0; 0 -8 4; 1 0 2]
% or [2, 8, 0; 0, -8, 4; 1, 0, 2]
```

(c) (1 pt) Using MATLAB syntax and *one line of code*, define vector $\mathbf{b}$.

```
b = [10; 6; 0]
```

(d) (2 pt) Based only on the following results, which of the following is(are) possible regarding the given system of linear equations? Select ALL that apply.

```
>> det(A)
        0

>> inv(A)
Warning: Matrix is singular to working precision.
```

- ■ There is no solution
- ☐ There is a unique solution
- ■ There are infinite solutions
- ☐ There are three solutions
- ☐ Can't determine anything about the solution based on the given information

(e) (1 pt) You realized that the third equation, $i_1 + 2i_3 = 0$, is incorrect and should be replaced by: $i_1 - i_2 - i_3 = 0$. Using MATLAB syntax and *one line of code*, and without redefining the entire matrix $\mathbf{A}$, only update the incorrect part of `A` based on the new equation.

```
A(3, :) = [1, -1, -1]
% A(3, 2:3) = [-1, -1]
% or equivalent
```

(f) (2 pt) After updating matrix `A`, you obtained the results shown below. Which of the following is(are) possible regarding the updated system of linear equations? Select ALL that apply.

```
>> det(A)
        56

>> inv(A)
       0.2143    0.1429    0.5714
       0.0714   -0.0357   -0.1429
       0.1429    0.1786   -0.2857
```

- ☐ There is no solution
- ■ There is a unique solution
- ☐ There are infinite solutions
- ☐ There are three solutions
- ☐ There is a solution but can't determine if it is unique or infinite

(g) (2 pt) Regardless of your answer to (f), which of the following MATLAB expressions can be used to solve the given system of linear equations? Select ALL that apply.

- ■ `A \ b`
- ☐ `A / b`
- ☐ `b \ A`
- ☐ `b / A`
- ■ `inv(A) * b`
- ☐ `inv(b) * A`
- ☐ `b * inv(A)`
- ☐ `pinv(b) * A`
- ☐ `(A' * A) * A' * b`

(h) (1 pt) Which is the most recommended method to solve any system of linear equations?

- ● `A \ b`
- ○ `A / b`
- ○ `b \ A`
- ○ `b / A`
- ○ `inv(A) * b`
- ○ `inv(b) * A`
- ○ `b * inv(A)`
- ○ `pinv(b) * A`
- ○ `(A' * A) * A' * b`

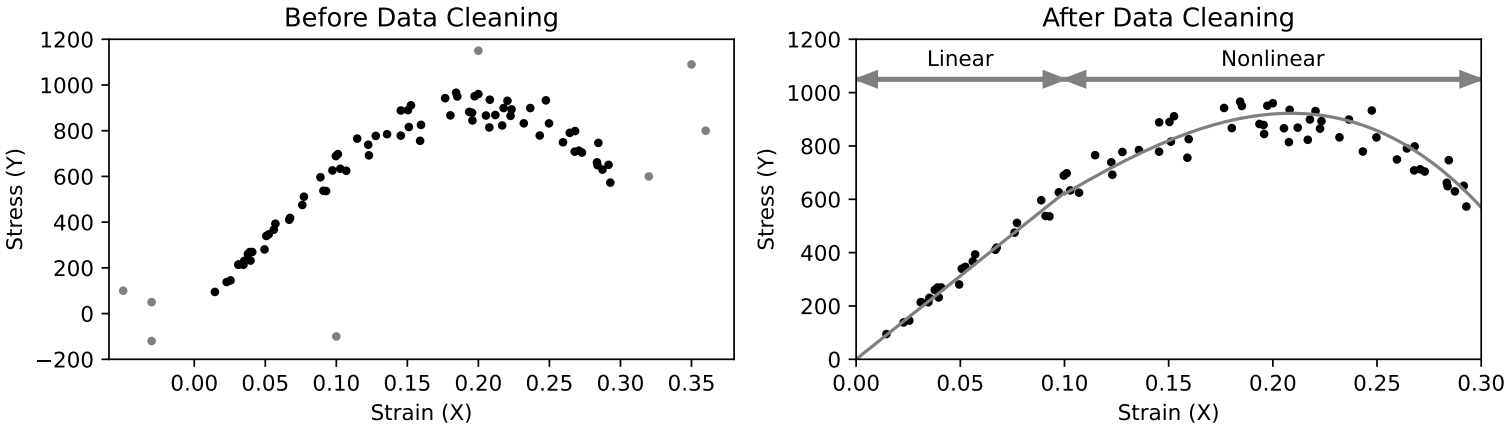
(i) (2 pt) What is(are) the benefits of solving a system of linear equations using linear algebra? Select ALL that apply.

- ■ Enables efficient solutions for systems with many equations and unknowns
- ■ Allows for concise representation using matrices and vectors
- ■ Permits easily identifying whether the system has a unique, no, or infinite solutions
- ☐ Only applicable to systems with a small number of equations and unknowns
- ☐ Provides exact solutions for all types of systems without exception

Question 7: A Regression Roller Coaster of Stress (10 points)

You are interested in understanding the physical properties of a new material, specifically its stress-strain relationship. You want to measure the force (i.e., stress) and deformation (i.e., strain) of the material when compressed. Since this is a new material, you performed extensive testing of the stress versus strain and the results are shown below. You imported the measurements into MATLAB as two **column** vectors, where vector `X` is the strain and `Y` is the corresponding stress.

The measurements include unrealistic values, which are shown using gray circles in the plot to the left. So first, you want to clean the data sets. After cleaning the data, you want to estimate the relationship between stress and strain. You will split the data into a linear range and a nonlinear range, and then fit a curve to the linear range only.



First, let's clean the data sets. For parts (a), (b), and (c), use MATLAB syntax and *one line of code* to write an expression that checks the given condition. The output should be an **array** that includes True for each measurement that satisfies the condition, and False otherwise.

(a) (1 pt) For each element in `X`, is the corresponding strain both positive and less than 0.3?

```
X > 0 & X < 0.3
```

(b) (1 pt) For each element in `Y`, is the corresponding stress both positive and less than 1000?

```
Y > 0 & Y < 1000
```

(c) (1 pt) For each measurement, is the strain both positive and less than 0.3 and is the stress both positive and less than 1000? Assign the result to a variable named `condition`.

```
condition = X > 0 & X < 0.3 & Y > 0 & Y < 1000
```

(d) (1 pt) Using MATLAB syntax and *one line of code*, write an expression that extracts all elements of `X` where the strain is both positive and less than 0.3 and the corresponding stress is both positive and less than 1000. Assign these elements to `strain`. You may want to use `condition`, which was defined in part (c).

```
strain = X(condition)
```

(e) (1 pt) Using MATLAB syntax and *one line of code*, write an expression that extracts all elements of `Y` where the strain is both positive and less than 0.3 and the corresponding stress is both positive and less than 1000. Assign these elements to `stress`. You may want to use `condition`, which was defined in part (c).

```
stress = Y(condition)
```

(f) (2 pt) Next, after cleaning the data, you want to extract data within the linear region. The linear region is defined where the strain is less than 0.1 (refer to the plot on the previous page). Complete the code below to extract the data for the linear region.

```
% strainL and stressL are the values for linear region (after cleaning)

strainL = strain(strain < 0.1)

stressL = stress(strain < 0.1)
```

(g) (3 pt) For the linear region, the relationship between stress and strain can be written as: $stress = E \times strain$, where E is an unknown constant value that depends on the material. Using linear regression and *one line of code*, estimate the value of E. Note that the stress should be 0 when strain is 0, so compute E accordingly! Assign the value to variable `E`.

```
E = strainL \ stressL
```